COLLEGE OF ENGINEERING & MANAGEMENT PUNNAPRA

Under Co-operative Academy of Professional Education
Estd. by Govt. of Kerala
VADACKAL(PO), ALAPPUZHA - 688003

LABORATORY RECORD

BTech Computer Science & Engineering

CS 431 Compiler Design Lab

Year 2021-2022

January 12, 2022

Contents

Title	Page No	Date	Signature
1 Lexical analyser	4		
2 E-closure	9		
3 Operator precedence parser	14		
4 Recursive Descent parser	19		
5 Shift reduce parser	23		
6 Constant propagation	28		
7 Intermediate code generation	31		
8 Conversion from three address code to 8086 assembly language instructions	35		
9 Lexical analyser using Lex tool	37		
10 Validate arithmetic expression using LEX and YACC tool	41		
11 Vaildate variable using LEX and YACC tool	45		
12 Calculator using LEX and YACC tool	48		

Program No:1
Date :29/09/2020

1. Lexical analyser

Aim

Design and implement lexical analyser for a given language using C and lexical analyzer should ignore redundant spaces, tabs and new lines.

Algorithm

Input: Read a C program.
Output: Print whether the lexemes are digit,keyword,identifier,special symbols or operators.
Step1: Start theprogram.
Step2: Declare all the file pointers and variables.
Step3: Read the input program and store it in the file f. Step4:
Repeat step 5 to 8 until the end of the file is reached. Step5:
Take each characters one by one .
Step6: Check whether the characters is digit,keyword,identifier,operators or special symbols.
Step7: Group the consecutive characters with similar properties.
Step8: Print the numbers,keywords,identifiers, operators and special symbols of the input program.
Step9: Stop.

Program

```
#include<string.h>
#include<ctype.h>
#include<stdio.h>
void main()
{
   FILE *f;
   char c,str[10];
   int n=0,i=0;
   printf("Enter the c program¥n"); f=fopen("input.txt","w");
```

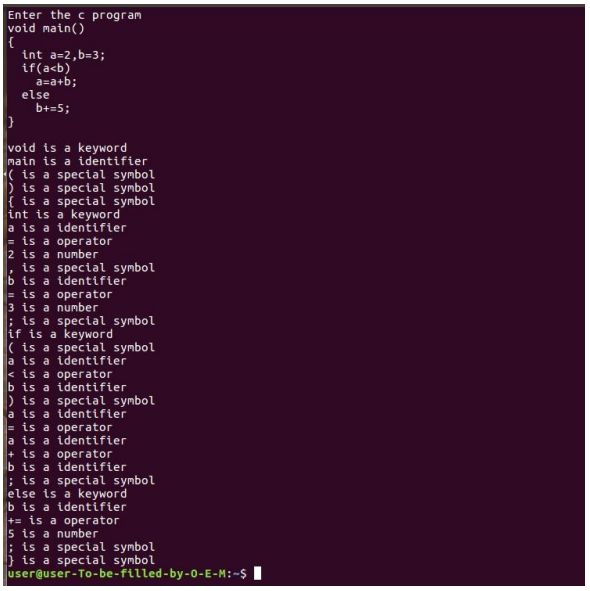
```
while((c=getchar())!=EOF)
{
   putc(c,f);
}
fclose(f);
f=fopen("input.txt","r");
while((c=getc(f))!=EOF)
{
   if(isdigit(c))
   {
       n=c-48;
       c=getc(f);
       while(isdigit(c))
       {
          n=n*10+(c-48);
          c=getc(f);
       }
       printf("¥n %d is a number ",n);
       ungetc(c,f);
   }
   else if(isalpha(c))
   {
       str[i++]=c;
       c=getc(f);
       while(isdigit(c)||isalpha(c)||c=='_'||c=='$')
       {
           str[i++]=c;
           c=getc(f);
       }
       str[i++]='¥0';
       if(strcmp("auto",str)==0||strcmp("break",str)==0||
          strcmp("case",str)==0||strcmp("char",str)==0||
          strcmp("const",str)==0||strcmp("continue",str)
          ==0||strcmp("default",str)==0||strcmp("do",str)
          ==0||strcmp("double",str)==0||strcmp("else",str
          )==0||strcmp("enum",str)==0||strcmp("extern",
          str)==0||strcmp("float",str)==0||strcmp("for",
          str)==0||strcmp("goto",str)==0||strcmp("if",str
          )==0||strcmp("int",str)==0||strcmp("long",str)
```

```
        ==0||strcmp("register",str)==0||strcmp("return
        ",str)==0||strcmp("short",str)==0||strcmp("
        signed",str)==0||strcmp("sizeof",str)==0||
        strcmp("static",str)==0||strcmp("struct",str)
        ==0||strcmp("switch",str)==0||strcmp("typedef",
        str)==0||strcmp("union",str)==0||strcmp("
        unsigned",str)==0||strcmp("void",str)==0||
        strcmp("volatile",str)==0||strcmp("while",str)
        ==0)
    {
        printf("¥n %s is a keyword",str);
    }
    else
    {
        printf("¥n %s is a identifier",str);
    }
    ungetc(c,f);
    i=0;
}
else if(c==' ' ||c=='¥t' ||c=='¥n')
{
    continue;
}
else if(c=='+' ||c=='-' ||c=='*' ||c=='/' ||c=='%
' ||c
  =='=' ||c=='>' ||c=='<' ||c=='!' ||c=='&' ||c=='|
  ' ||c
  =='^')
{
    str[i++]=c;
    c=getc(f);
    while(c=='+' ||c=='-' ||c=='*' ||c=='/' ||c=='%
    ' ||c
       =='=' ||c=='>' ||c=='<' ||c=='!' ||c=='&' ||
       c=='|' || c=='^')
    {
        str[i++]=c;
        c=getc(f);
    }
```

```
str[i++]=' ¥0' ;
if(strcmp("+",str)==0||strcmp("-",str)==0||strcmp
   ("*",str)==0||strcmp("/",str)==0||strcmp(" %",
```

```
            str)==0||strcmp("=",str)==0||strcmp("!",str)
            ==0||strcmp("&",str)==0||strcmp("|",str)==0||
            strcmp("^",str)==0||strcmp(">",str)==0||strcmp
            ("<",str)==0||strcmp("+=",str)==0||strcmp("-=",
            str)==0||strcmp("*=",str)==0||strcmp("/=",str)
            ==0||strcmp(" %=",str)==0||strcmp("==",str)==0||
            strcmp("!=",str)==0||strcmp(">=",str)==0||
            strcmp("<=",str)==0||strcmp("&&",str)==0||
            strcmp("||",str)==0||strcmp("<<",str)==0||
            strcmp(">>",str)==0||strcmp("++",str)==0||
            strcmp("--",str)==0)
        {
            printf("¥n %s is a operator",str);
        }
        else
        {
            printf("¥n %s is a invalid operator",str);
        }
        ungetc(c,f);
        i=0;
    }
    else
    {
        printf("¥n %c is a special symbol",c);
    }
  }
  printf("¥n");
  fclose(f);
}
```

Output



Result

The program is executed and the output is obtained and verified.

Program No:2
Date :06/10/2020

2. E-closure

Aim

Write a program to find E closure for all states of any given NFA with E transition.

Algorithm

Input: States,symbols and transitions of a NFA.
Output: Print the epsilon closure of the given NFA.
Step1: Start the program.
Step2: Declare all the variables and structures.
Step3: Read the number of states, symbols and transitions.
Step4: Find the epsilon transition of each state and store the result in z.
Step5: Merge the epsilon transition of each states in the z and store the result in r.
Step6: Remove the repeated values from the r and hence the epsilon closure is obtained.
Step7: Print the epsilon closure of each state.
Step8: Stop.

Program

```
#include<string.h>
#include<ctype.h>
#include<stdio.h>
struct states
{
  int ss;
  int sd;
  char a[3];
}st[25];
void main()
{
  int n,na,ns,i,j,z[15][15],r[30][30],t=0,t1=0,t2=0,k;
```

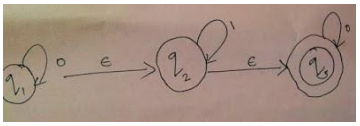
```
printf("Enter the number of states¥n");
scanf("%d",&ns);
printf("Enter the number of alphabets including epsilon¥n
   ");
scanf("%d",&na);
printf("Enter the number of transitions¥n");
scanf("%d",&n);
printf("¥nHERE THE STATES STARTS FROM 1¥n");
for(i=0;i<n;i++)
{
    printf("Enter the source of %dth transitions ",i+1);
    scanf("%d",&st[i].ss);
    printf("Enter the symbol of %dth transitions ",i+1);
    scanf("%s",st[i].a);
    printf("Enter the destination of %dth transitions ",i
       +1);
    scanf("%d",&st[i].sd);
}
for(j=0;j<ns;j++)
{

    z[j][t]=j+1;
    t=1;
    for(i=0;i<n;i++)
    {

       if((st[i].ss==(j+1)) &&  (strcmp("e",st[i].a)==0))
       {

           z[j][t]=st[i].sd;
           t=t+1;
       }
    }
    z[j][t]=0;
    t=0;
}

for(j=0;j<ns;j++)
{
```

```
  t=0;
  t2=0;
  while(z[j][t]>0)
  {

     if(z[j][t]==j+1)
     {
        r[j][t2]=z[j][t];
        t2=t2+1;
     }
     else
     {
         t1=0;
         k=z[j][t];
         while(z[k-1][t1]>0)
         {
             r[j][t2]=z[k-1][t1];
             t1=t1+1;
             t2=t2+1;
         }
     }
     t=t+1;
  }
  r[j][t2]=0;
}
for(j=0;j<ns;j++)
{
  t=0;
  t2=0;
  while(r[j][t]>0)
  {

     if(r[j][t]==j+1)
     {
        z[j][t2]=r[j][t];
        t2=t2+1;
     }
     else
     {
```

```
            t1=0;
            k=r[j][t];
            while(r[k-1][t1]>0)
            {
                z[j][t2]=r[k-1][t1];
                t1=t1+1;
                t2=t2+1;
            }
        }
        t=t+1;
    }
    z[j][t2]=0;
}

for(j=0;j<ns;j++)
{
    t=0;
    while(z[j][t]>0)
    {
        t1=t+1;
          while(z[j][t1]>0)
          {
              if(z[j][t]==z[j][t1])
              {
                  t2=t1;
                  while(z[j][t2]>0)
                  {
                      z[j][t2]=z[j][t2+1];
                      t2=t2+1;
                  }
              }
              t1=t1+1;
          }
       t=t+1;
    }
}

for(j=0;j<ns;j++)
{
```

```
        t=0;
        printf("e closure of q %d is {",j+1);
         while(z[j][t]>0)
         {
            printf("q %d ",z[j][t]);
            t=t+1;
         }
         printf("}¥n");
    }

}
```

Input



Output

```
user@user-To-be-filled-by-O-E-M:~$ gcc test3.c
user@user-To-be-filled-by-O-E-M:~$ ./a.out
Enter the number of states
3
Enter the number of alphabets including epsilon
3
Enter the number of transitions
5

HERE THE STATES STARTS FROM 1
Enter the source of 1th transitions 1
Enter the symbol of 1th transitions 0
Enter the destination of 1th transitions 1
Enter the source of 2th transitions 1
Enter the symbol of 2th transitions e
Enter the destination of 2th transitions 2
Enter the source of 3th transitions 2
Enter the symbol of 3th transitions 1
Enter the destination of 3th transitions 2
Enter the source of 4th transitions 2
Enter the symbol of 4th transitions e
Enter the destination of 4th transitions 3
Enter the source of 5th transitions 3
Enter the symbol of 5th transitions 0
Enter the destination of 5th transitions 3
e closure of q1 is {q1 q2 q3 }
e closure of q2 is {q2 q3 }
e closure of q3 is {q3 }
user@user-To-be-filled-by-O-E-M:~$ gcc test3.c
```

Result

The program is executed and the output is obtained and verified.

Program No:3
Date :19/10/2020

3. Operator precedence parser

Aim

Write a program to develop an operator precedence parser for a given language.

Algorithm

Input: Read thestring
Output: Print the whether the string is accepted or not
Step1 : Start.
Step2 : Define the precedence table and the grammar globally. Step3 : Input the string and store it in 'input'.
Step4 : Append dollar symbol to the last.
Step5 : Compute the string length, initialize i=0.
Step6 : Repeat the step 6 to 9 until the end of the input string. Step7 : Move the ith element of input to the stack.
Step8 : Print stack and input string.
Step9 : If the top element of the stack is equal to ">" then reduce the stack element with the corresponding stack element else increment i.
Step10 : If "*E*" is in stack then print that the string is accepted else not.
Step11 : Stop.

Program

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
char *input;
int i=0;
char  lasthandle[6],stack[50],handles[][5]={")E(","E*E","E+E
   ","i","E^E","E-E","E/E"};
int  top=0,l;
char  prec[9][9]={
```

```
            /*stack + - * / ^ i ( ) $ */

            /* + */ ' >' ,
            ' >' ,' <' ,' <' ,' <' ,' <' ,' <' ,' >' ,' >' ,

            /* - */ ' >' ,
            ' >' ,' <' ,' <' ,' <' ,' <' ,' <' ,' >' ,' >' ,

            /* * */ ' >' ,
            ' >' ,' >' ,' >' ,' <' ,' <' ,' <' ,' >' ,' >' ,

            /* / */ ' >' ,
            ' >' ,' >' ,' >' ,' <' ,' <' ,' <' ,' >' ,' >' ,

            /* ^ */ ' >' ,
            ' >' ,' >' ,' >' ,' <' ,' <' ,' <' ,' >' ,' >' ,

            /* i */ ' >' ,
            ' >' ,' >' ,' >' ,' >' ,' e' ,' e' ,' >' ,' >' ,

            /* ( */ ' <' ,
            ' <' ,' <' ,' <' ,' <' ,' <' ,' <' ,' >' ,' e' ,

            /* ) */ ' >' ,
            ' >' ,' >' ,' >' ,' >' ,' e' ,' e' ,' >' ,' >' ,

            /* $ */ ' <' ,
            ' <' ,' <' ,' <' ,' <' ,' <' ,' <' ,' <' ,' >' ,

               };

int getindex(char c)
{
        switch(c)
        {
                case ' +' :retu 0;
                          rn
                case     ' -    1;
                     ' :return
                case ' *' :retu 2;
```

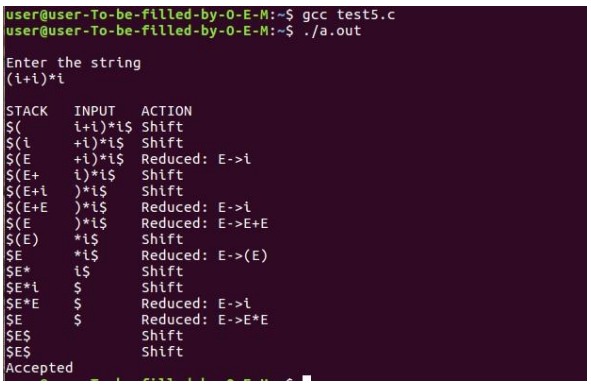
```
                rn
            case ' /' :retu 3;
                rn
            case ' ^' :retu 4;
                rn
            case ' i' :retu 5;
                rn
            case ' (' :retu 6;
                rn
            case ' )' :retu 7;
                rn
            case ' $' :retu 8;
        }           rn
}
```

```
int shift()
{
        stack[++top]=*(input+i++);
        stack[top+1]='¥0';
}


int reduce()
{
        int i,len,found,t;
        for(i=0;i<7;i++)//selecting  handles
        {
                len=strlen(handles[i]);
                if(stack[top]==handles[i][0]&&top+1>=len)
                {
                        found=1;
                        for(t=0;t<len;t++)
                        {
                                 if(stack[top-t]!=handles[i][t
                                   ])
                                {
                                        found=0;
                                        break;
                                }
                        }
                        if(found==1)
                        {
                                stack[top-
                                t+1]='E'; top=top-
                                t+1;
                                strcpy(lasthandle,handles[i]);
                                stack[top+1]='¥0';
                                return 1;
                        }
                }
        }
        return 0;
}
```

```
void dispstack()
{
        int j;
        for(j=0;j<=top;j++)
                printf("%c",stack[j]);
}



void dispinput()
{
        int j;
        for(j=i;j<l;j++)
                printf("%c",*(input+j));
}



void main()
{
        int j;
        input=(char*)malloc(50*sizeof(char));
        printf("¥nEnter the string¥n");
        scanf(" %s",input);
        input=strcat(input,"$");
        l=strlen(input);
        strcpy(stack,"$");
        printf("¥nSTACK¥tINPUT¥tACTION");
        while(i<=l)
        {
                shift();
                printf("¥n");
                dispstack();
                printf("¥t");
                dispinput();
                printf("¥tShift");
                if(prec[getindex(stack[top])][getindex(input
                   [i])]==' >' )
```

```
                {
                        while(reduce())
                        {
                                printf("¥n");
                                dispstack();
                                printf("¥t");
                                dispinput();
                                if(strcmp(lasthandle,")E(")
                                   ==0)
                                {
                                        strcpy(lasthandle,"(E)
                                           ");
                                }
                                printf("¥tReduced: E-> %s",
                                   lasthandle);
                        }
                }
        }
        if(strcmp(stack,"$E$")==0)
                printf("¥nAccepted¥n");
        else
            printf("¥nNot Accepted¥n");
}
```

Output



Result

The program is executed and the output is obtained and verified.

Program No:4
Date :17/10/2020

4. Recursive Descent parser

Aim

Write a program to construct Recursive Descent parser for an expression.

Algorithm

Input: Read theexpression
Output: Print the whether the expression is valid or not
Step1 : Start.
Step2 : Input the expression and store it in 'expr'.
Step3 : Initialize and set flag, count to 0.
Step4 : Make functions for every production.
Step5 : Call the function of the starting symbol.
Step6 : Take each element in the expression and check whether the element is present or not. If it is present increment count else set flag to 1 Step7 : If length of the expression is equal to count and the flag is equal to 0 print valid else invalid.
Step8 : Stop.

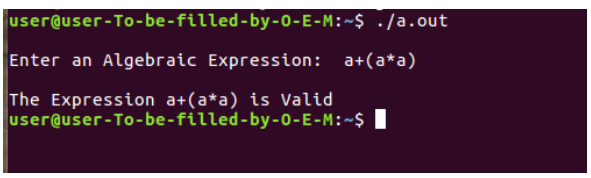
Program

```
#include<stdio.h>
#include<ctype.h>
#include<string.h>
void Td();
void Ed();
void E();
void F();
void  T();
char expr[10];
int c, flag;

int main()
{
      c = 0;
```

```
        flag = 0;
        printf("¥nEnter an Algebraic Expression:¥t");
        scanf("%s", expr);
        E();
        if((strlen(expr) == c) && (flag == 0))
        {
                printf("¥nThe  Expression %s is Valid¥n", expr);
        }
        else
        {
                printf("¥nThe  Expression %s is Invalid¥n", expr)
                   ;
        }
}

void E()
{
        T();
        Ed();
}

void T()
{
        F();
        Td();
}

void Td()
{
        if(expr[c] == ' *' )
        {
                c++;
                F();
                Td();
        }
}

void F()
{
```

```
        if(islower(expr[c]))
        {
                c++;
        }
        else if(expr[c] == '(')
        {
                c++;
                E();
                if(expr[c] == ')')
                {
                        c++;
                }
                else
                {
                        flag = 1;
                }
        }
        else
        {
                flag = 1;
        }
}

void Ed()
{
        if(expr[c] == '+')
        {
                c++;
                T();
                Ed();
        }
}
```

Output



Result

The program is executed and the output is obtained and verified.

Program No:5
Date :29/12/2020

5. Shift reduce parser

Aim

Write a program to construct a shift reduce parser for a given language.

Algorithm

Input: Read thestring
Output: Print the whether the string is accepted or not
Step1 : Start.
Step2 : Define 4 single dimensional arrays globally.
Step3 : Input the string and store it in array 'a'.
Step4 : Compute the string length and store it in 'c'.
Step5 : initializei=0.
Step6 : Repeat the step 6 to 10 until the end of the input string. Step7 : Move the ith element of input to the stack.
Step8 : Print stack and input string.
Step9 : If the top element of the stack is equal to the predefined grammar then reduce the stack element with the corresponding stack element else increment i and print shift.
Step10 : If "*E*" is in stack then print that the string is accepted else not.
Step11 : Stop.

Program

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
int z = 0, i = 0, j = 0, c = 0;
char a[16], h1[20], stack[15], h2[10];
void check()
{
    strcpy(h1,"REDUCE TO E -> ");
    for(z=0;z<c;z++)
    {
        if(stack[z] == ' a' )
```

```
    {
        printf("%sa", h1);
        stack[z] = ' E' ;
        stack[z + 1] = ' ¥0' ;
        printf("¥n$ %¥t %$¥t", stack, a);
    }
}
for(z=0;z<c;z++)
{
    if(stack[z] == ' (' && stack[z + 1] == ' E' &&
       stack[ z + 2] == ' )' )
    {
        printf(" %s(E)", h1);
        stack[z]=' E' ;
        stack[z + 1]=' ¥0' ;
        stack[z + 2]=' ¥0' ;
        printf("¥n$ %¥t %$¥t", stack, a);
        i = i - 2;
    }
}
for(z=0;z<c;z++)
{
    if(stack[z] == ' E' && stack[z + 1] == ' *' &&
       stack[ z + 2] == ' E' )
    {
        printf("%sE*E", h1);
        stack[z] = ' E' ;
        stack[z + 1] = ' ¥0' ;
        stack[z + 2] = ' ¥0' ;
        printf("¥n$ %¥t %$¥t", stack, a);
        i = i - 2;
    }

}

for(z=0;z<c;z++)
{
    if(stack[z] == ' E' && stack[z + 1] == ' +' &&
       stack[ z + 2] == ' E' )
```

```
        {
            printf("%sE+E", h1);
            stack[z]=' E' ;
            stack[z + 1]=' ¥0' ;
            stack[z + 2]=' ¥0' ;
            printf("¥n$ %¥t %$¥t", stack, a);
            i = i - 2;
        }
    }
    for(z=0;z<c;z++)
    {
        if(stack[z] == ' E'  && stack[z + 1] == ' -'  &&
           stack[ z + 2] == ' E' )
        {
            printf("%sE-E", h1);
            stack[z]=' E' ;
            stack[z + 1]=' ¥0' ;
            stack[z + 2]=' ¥0' ;
            printf("¥n$ %¥t %$¥t", stack, a);
            i = i - 2;
        }
    }
    for(z=0;z<c;z++)
    {
        if(stack[z] == ' E'  && stack[z + 1] == ' /'  &&
           stack[ z + 2] == ' E' )
        {
            printf("%sE/E", h1);
            stack[z]=' E' ;
            stack[z + 1]=' ¥0' ;
            stack[z + 2]=' ¥0' ;
            printf("¥n$ %¥t %$¥t", stack, a);
            i = i - 2;
        }
    }

    return ;
}
int main()
```

```
{
    printf("GRAMMAR is -¥nE->a ¥nE->E*E ¥nE->E+E ¥nE->E-E ¥
       nE->E/E ¥nE->(E)¥n");
    printf("enter the string¥n");
    gets(a);
    c=strlen(a);
    strcpy(h2,"SHIFT");
    printf("¥nstack ¥t input ¥t¥t action");
    printf("¥n$¥t %s$¥t", a);
    for(i = 0; j < c; i++,j++)
    {
        printf("%s", h2);
        stack[i] = a[j];
        stack[i + 1] = ' ¥0' ;
        a[j]=' ' ;
        printf("¥n$ %s¥t %s$¥t", stack, a);
        check();

    }
    if(stack[0] == ' E'  && stack[1] == ' ¥0' )
        printf("Accept¥n");
    else
        printf("Reject¥n");
}
```

Output

```
user@user-To-be-filled-by-O-E-M:~$ ./a.out
GRAMMAR is -
E->a
E->E*E
E->E+E
E->E-E
E->E/E
E->(E)
enter the string
(a+a)*a

stack    input          action
$       (a+a)*a$        SHIFT
$(       a+a)*a$        SHIFT
$(a       +a)*a$        REDUCE TO E -> a
$(E       +a)*a$        SHIFT
$(E+       a)*a$        SHIFT
$(E+a       )*a$        REDUCE TO E -> a
$(E+E       )*a$        REDUCE TO E -> E+E
$(E         )*a$        SHIFT
$(E)         *a$        REDUCE TO E -> (E)
$E           *a$        SHIFT
$E*           a$        SHIFT
$E*a           $        REDUCE TO E -> a
$E*E           $        REDUCE TO E -> E*E
$E             $        Accept
user@user-To-be-filled-by-O-E-M:~$
```

Result

The program is executed and the output is obtained and verified.

Program No:6
Date :29/12/2020

6. Constant propagation

Aim

Write a program to perform constant propagation.

Algorithm

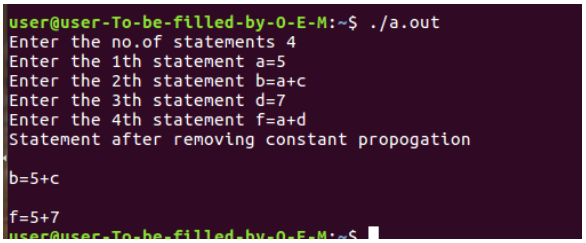
Input:Read an input expressions.
Output:Perform constant propagation and print the result.
Step1 : Start.
Step2 : Declare the variable and two dimensional array.
Step3 : Read no.of expression and repeat the step 4 to step 8 until all the expression are processed.
Step4 : Read each expression and checks whether the RHS of a expression is a digit or not.
Step5 : If it is a digit then store that digit into a variable 'f' and the variable name into 't'.
Step6 : If the next expression variable name is same as the previous then ignore that expression.
Step7 : Check whether the next expression matches with the value in the variable 'f' then replace that with 't' otherwise process till the expression is completed then print the expression itself.
Step8 : Stop.

Program

```
#include<stdio.h>
#include<string.h>
void main()
{
        int n,i,j,k,f;
        char s[10][10],t;
        printf("Enter the no.of statements ");
        scanf("%d",&n);
        for(i=0;i<n;i++)
        {
```

```
            printf("Enter  the %dth statement ",i+1);
            scanf("%s",s[i]);
        }
        for(i=0;i<n;i++)
        {
            if(isdigit(s[i][2]) && s[i][3]=='¥0')
            {
                f=s[i][2];
                t=s[i][0];
                s[i][0]='¥0';
                for(j=i+1;j<n;j++)
                {
                    if(s[j][0]==t)
                    {

                        break;
                    }
                    k=2;
                    while(s[j][k]!='¥0')
                    {
                        if(s[j][k]==t)
                        {
                            s[j][k]=f;
                        }
                        k=k+1;
                    }
                }
            }
        }
        printf("Statement after removing constant
           propogation¥n");
        for(i=0;i<n;i++)
        {
            printf("%s¥n",s[i]);
        }
}
```

Output



Result

The program is executed and the output is obtained and verified.

Program No:7 Date
: 12/01/2021

7. Intermediate code generation

Aim

Implement intermediate code generation for simple expressions

Algorithm

Input: Input theexpression
Output: Print the intermediate code of the expression
Step1 : Start.
Step2 : Input the expression and store it in an array 'a'. Step3 :
Find the length of the expression and store it in 'n'. Step4 :
Initializej=0
Step5 : if j< n goto step 6 else goto step 25.
Step6 : Initializei=0
Step7 : If i<n goto step 8 else goto step 15
Step8 : If a[i] is equal to '*' or a[i] is equal to '/' goto step 9 else goto step 7.
Step9 : Assignd=i-1
Step10 : If a[d]is equal to ' ' then goto step 11 else goto step12.
Step11 : decrement d by 1 and goto step 10.
Step12 : Print the values stored in c equal to the values in a[d],a[i],a[i+1].
Step13 : Replace the value in a[d] by c , a[i] by space, a[i+1] by space and increment c.
Step14 : Increment i by 1 and goto step 7. Step15
: Initialize i=0.
Step16 : If i< n goto step 17 else goto step 15.
Step17 : If a[i] is equal to '+' or a[i] is equal to '-' goto step 18 else goto step 16.
Step18 : Assignd=i-1.
Step19 : If(a[d]is equal to ' ' then goto step 20 else goto step21.
Step20 : decrement d by 1 and goto step 19.
Step21 : Print as the values stored in c equal to the values in a[d],a[i],a[i+1].
Step22 : Replace the value in a[d] by c , a[i] by space, a[i+1] by space and increment c.

Step23 : Increment i by 1 and goto step 16.
Step24 : Increment j by 1 and goto step 5. Step25
: Stop.

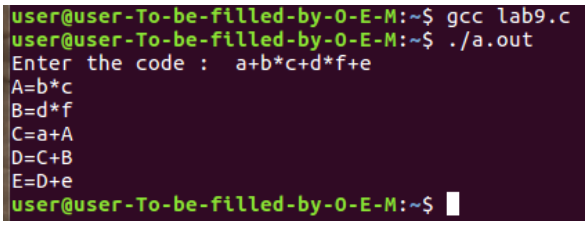
Program

```
#include<stdio.h>
#include<string.h>
void main()
{
    int j=0,i=0,n=0,d=0;
    char a[20],c=' A' ;
    printf("Enter the code : ");
    scanf("%s",a);
    while(a[i]!=' ¥0' )
    {

        n=n+1;
        i=i+1;

    }
    for(j=0;j<n;j++)
    {

        for(i=0;i<n;i++)
        {

            if(a[i]==' *' ||a[i]==' /' )
            {
                d=i-1;
                while(a[d]==' ' )
                {

                    d=d-1;

                }
                if(d!=(i-1))
                {
                    printf("%c=%c%c%c¥n",c,a[d],a[i],a[i+1]);
```

```
            a[d]=c;
            a[i]=' ';
            a[i+1]=' ';
            c=c+1;
            continue;
        }
        else if(d==(i-1))
        {
            printf("%c=%c%c%c¥n",c,a[i-1],a[i],a[i
               +1]);
            a[i-1]=c;
            a[i]=' ';
            a[i+1]=' ';
            c=c+1;
            continue;
        }
    }
}
 for(i=0;i<n;i++)
{

    if(a[i]=='+'||a[i]=='-')
    {
        d=i-1;
        while(a[d]==' ')
        {

            d=d-1;

        }
        if(d!=(i-1))
        {
            printf("%c=%c%c%c¥n",c,a[d],a[i],a[i+1]);
            a[d]=c;
            a[i]=' ';
            a[i+1]=' ';
            c=c+1;
            continue;
        }
```

```
                else if(d==(i-1))
                {
                    printf("%c=%c%c%c¥n",c,a[i-1],a[i],a[i
                       +1]);
                    a[i-1]=c;
                    a[i]=' ';
                    a[i+1]=' ';
                    c=c+1;
                    continue;
                }
            }
        }

    }
}
```

Output



Result

The program is executed and the output is obtained and verified.

Program No:8 Date
: 12/01/2021

8. Conversion from three address code to 8086 assembly language instructions

Aim

Write a c program to implement the back end of the compiler to take the three address code and produced 8086 assembly language instructions that can be assembled and the run using an assembler. The target assembly instructions can be simple mov, add etc

Algorithm

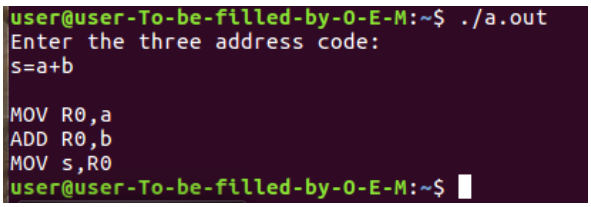
Input: Input the three address code
Output: Print the corresponding 8086 assembly language instruction
Step1 : Start.
Step2 : Input the three address code and store it in 'a'.
Step3 : Store a[3] in 'ch'
Step4 : If 'ch is equal to '+' then print MOV R0 a[2] , ADD R0 a[4] , MOV a[0] R0 and goto step 9.
Step5 : If 'ch is equal to '-' then print MOV R0 a[2] , SUB R0 a[4] , MOV a[0] R0 and goto step 9.
Step6 : If 'ch is equal to '*' then print MOV R0 a[2] , MUL R0 a[4] , MOV a[0] R0 and goto step 9.
Step7 : If 'ch is equal to '/' then print MOV R0 a[2] , DIV R0 a[4] , MOV a[0] R0 and goto step 9.
Step8 : Print INVALID
Step9 : Stop.

Program

```
#include<stdio.h>
#include<string.h>
void main()
{
 char a[10],ch;
 printf("Enter the three address code:¥n");
 gets(a);
```

```
 ch=a[3];
 switch(ch)
 {
     case '+'  :
               printf("¥nMOV R0,%c",a[2]);
               printf("¥nADD R0,%c",a[4]);
               printf("¥nMOV %c,R0¥n",a[0]);
               break;
     case '-'  :
               printf("¥nMOV R0,%c",a[2]);
               printf("¥nSUB R0,%c",a[4]);
               printf("¥nMOV %c,R0¥n",a[0]);
               break;
     case '*'  :
               printf("¥nMOV R0,%c",a[2]);
               printf("¥nMUL R0,%c",a[4]);
               printf("¥nMOV %c,R0¥n",a[0]);
               break;
     case '/'  :
               printf("¥nMOV R0,%c",a[2]);
               printf("¥nDIV R0,%c",a[4]);
               printf("¥nMOV %c,R0¥n",a[0]);
               break;
     default : printf("INVALID¥n") ;
 }

}
```

Output



Result

The program is executed and the output is obtained and verified.

Program No:9
Date :13/01/2021

9. Lexical analyser using Lex tool

Aim

Write a program to implement lexical analyser using Lex tool.

Algorithm

Input: A C-program
Output: Print the preprocessor,keyword,identifier,function, operators etc in the C program.
Step1 : Start.
Step2 : Define the definition section if any.
Step3 : Define the rule section with regular expression for preprocessor,keyword,identifier,function,operators etc.
Step4 : Define the subroutine part.
Step5 : Store the file containing the C program to a file pointer.
Step6 : Store it to yyin.
Step7 : call yylex().
Step8 : invoke yywrap() to check end of the file. Step9 : Stop

Program

```
identifier [a-zA-Z][a-zA-Z0-9]*
%%
#.* { printf("¥n %s is a PREPROCESSOR DIRECTIVE",yytext);}
int |
float |
char |
double |
while |
for |
do |
if |
break |
```

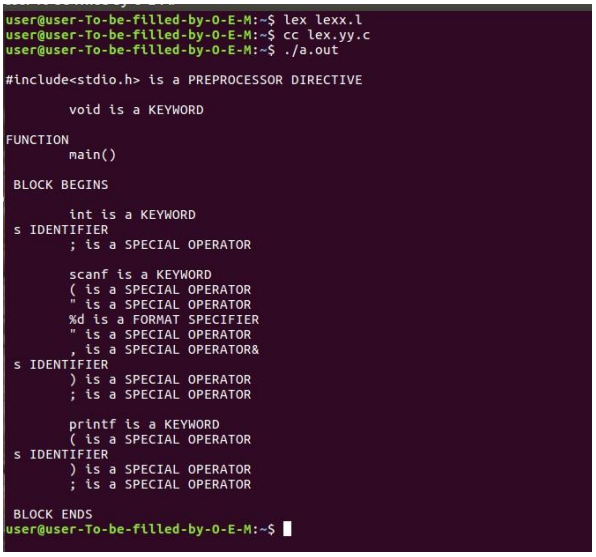
```
continue |
void |
switch |
case |
long |
struct |
const |
typedef |
return |
else |
printf |
goto {printf("¥n¥t %s is a KEYWORD",yytext);}
{identifier}¥(¥) {printf("¥n¥nFUNCTION¥n¥t %s",yytext);}
¥{ { printf("¥n BLOCK BEGINS");}
¥} { printf("¥n BLOCK ENDS");}
{identifier}(¥[[0-9]*¥])? { printf("¥n %s IDENTIFIER",
   yytext);}
[0-9]+ {printf("¥n¥t %s is a NUMBER",yytext);}
= {printf("¥n¥t %s is an ASSIGNMENT OPERATOR",yytext);}
¥<= |
¥>= |
¥< |
== |
¥!= |
¥> { printf("¥n¥t %s is a RELATIONAL OPERATOR",yytext);}
¥+ |
¥- |
¥* |
¥% |
¥/ {printf("¥n¥t %s is a ARITHMETIC OPERATOR",yytext);}
¥; |
¥: |
¥, |
¥" |
¥¥ |
¥( |
¥) |
¥'  {printf("¥n¥t %s is a SPECIAL OPERATOR",yytext);}
¥¥n |
```

```
¥¥t |
¥¥0 {printf("¥n¥t %s is a ESCAPE SEQUENCE",yytext);}
¥%s |
¥%f |
¥%d |
¥%c  {printf("¥n¥t%s is a FORMAT SPECIFIER",yytext);}
%%
int main()
{
 FILE *file;
 file = fopen("prg.c","r");
 if(!file)
 {
  printf("could not open file ¥n");
  exit(0);
 }
 yyin = file;

 yylex();
return 0;
}
int yywrap()
{
return 1;

}
```

prg.c

```
#include<stdio.h>
void main()
{
  int   s;
  scanf("%d",&s);
  printf(s);
}
```

Output



Result

The program is executed and the output is obtained and verified.

Program No: 10.a
Date :13/01/2021

10. Validate arithmetic expression using LEX and YACC tool

Aim

Write a program to recognise a valid arithmetic expression that use the operator +, -, * and / using LEX and YACC tool.

Algorithm

```
Input: Read an arithmetic expression
Output: Print whether the expression is valid or not.
Step1 : Start.
Step2 : Define the definition section of lex and yacc if any.
Step3 : Define the rule section of lex with  numbers.
%%
[0-9]+ { yylval=atoi(yytext);
         return NUMBER;

       }
%%
Step4 : Define the rule section of yacc with arithmetic
   operators.
%%
 A:E{}
 E:E' +' E
|E' -' E
|E' *' E
|E' /' E
|' (' E' )'
| NUMBER
;
%%
```

Step5 : Define the main function in yacc such that, print valid if the expression is valid.
Step6 : Call yyerror() in yacc if the expression is invalid .
Step7 : call yyparse().
Step8 : Invoke yywrap() in lex to check the end.
Step9 : Stop

Program

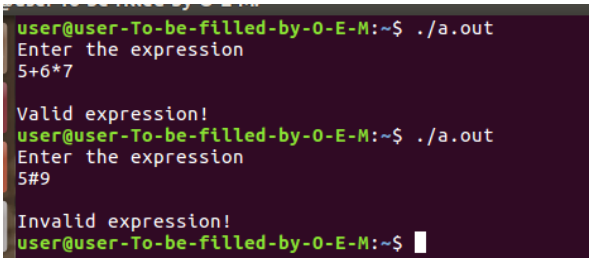
expval.l

```
%{
    /* Definition section*/
    #include "y.tab.h"
    extern yylval;
%}

%%
[0-9]+ {  yylval=atoi(yytext);
          return NUMBER;

       }

 [¥t]+ ;

¥n { return 0; }
. { return yytext[0]; }

%%
int yywrap()

{

return 1;

}
```

expval.y

```
%{
  int valid=0;
#include<stdio.h>
%}

%token NUMBER
%left ' +'
' -'
%left ' *'  ' /'
%%
 A:E{}
 E:E' +' E
|E' -' E
|E' *' E
|E' /' E
|' (' E' )'
| NUMBER

;
%%

    int main()
 {
    printf("Enter the expression¥n");
    yyparse();
    if(valid==0)
     printf("¥nValid expression!¥n");

 }
 int yyerror()

{
    valid=1;
    printf("¥nInvalid expression!¥n");
    return 0;

}
```

Output



Result

The program is executed and the output is obtained and verified.

Program No: 10.b
Date :13/01/2021

11. Vaildate variable using LEX and YACC tool

Aim

Write a program to recognise a valid variable which starts with letter followed by any number of letters and digits.

Algorithm

```
Input: Read an input variable.
Output: Print whether the variable is valid or not.
Step1 : Start.
Step2 : Define the definition section of lex and yacc if any.
Step3 : Define the rule section of lex with alphabet followed
   by digits or alphabets.
%%

[a-zA-Z_][a-zA-Z_0-9]* return letter;

%%
Step4 : Define the rule section of yacc with letter from the
   lex.
%%

start : s

s : letter ;

%%
Step5 : Define the main function in yacc such that, print
   variable if the variable is valid.
Step6 : Call yyparse().
Step7 : Call yyerror() in yacc if the variable is invalid.
Step8 : Invoke yywrap() in lex to check the end .
Step9 : Stop
```

Program

validvar.l

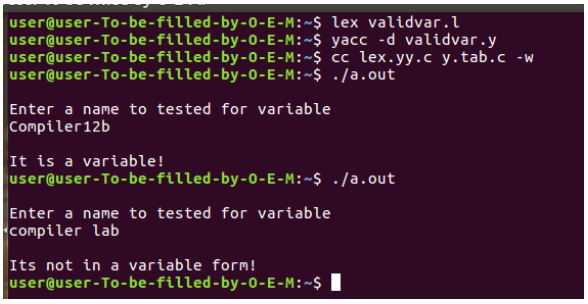
```
%{
    #include "y.tab.h"
%}

%%
[a-zA-Z_][a-zA-Z_0-9]* return letter;
. return yytext[0];
¥n return 0;
%%
int yywrap()
{
return 1;
}
```

validvar.y

```
%{

    #include<stdio.h>

    int valid=1;

%}

%token letter

%%

start : s

s : letter ;

%%

int yyerror()
{
    printf("¥nIts not in a variable form!¥n");
```

```
    valid=0;
    return 0;
}

int main()
{
    printf("¥nEnter a name to tested for variable ¥n");
    yyparse();
    if(valid)

    {
        printf("¥nIt is a variable!¥n");
    }
}
```

Output



Result

The program is executed and the output is obtained and verified.

Program No: 10.c
Date :13/01/2021

12. Calculator using LEX and YACC tool

Aim

Write a program to implement a calculator using LEX and YACC tool.

Algorithm

```
Input: Read an arithmetic expression
Output: Print whether the expression is valid or not.
Step1 : Start.
Step2 : Define the definition section of lex and yacc if any.
Step3 : Define the rule section of lex with  numbers.
%%
[0-9]+ { yylval=atoi(yytext);
         return NUMBER;

       }
%%
Step4 : Define the rule section of yacc with arithmetic
   operators and print the result by performing the
   operation.
%%
A:E
{       printf("¥nResult= d¥n", $$);
        return 0;          %

}
E:E' +' E {$$=$1+$3;}
|E' -' E {$$=$1-$3;}
|E' *' E {$$=$1*$3;}
|E' /' E {$$=$1/$3;}
|E'%' E {$$=$1 % $3;}
|' (' E' )'   {$$=$2;}
| NUMBER {$$=$1;}
```

```
;
%%
Step5 : Define the main function in yacc and call yyparse()
   .
Step6 : Call yyerror() in yacc if the expression is invalid
   .
Step7 : Invoke yywrap() in lex to check the end.
Step8 : Stop
```

Program

cal.l

```
%{
  #include<stdio.h>
  #include"y.tab.h"
  extern int yylval;
%}
%%
[0-9]+ { yylval=atoi(yytext);
          return NUMBER;

       }
[¥t] ;

[¥n] return 0;
. return yytext[0];

%%

int yywrap()
{
 return 1;
}
```

cal.y

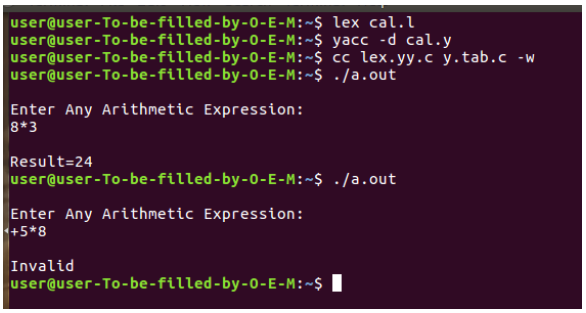
```
%{
#include<stdio.h>
int flag=0;
```

```
%}
%token NUMBER

%left ' +'

' -'

%left ' *'  ' /'  ' %'

%left ' ('  ' )'
%%

A:E
{
        printf("¥nResult= %d¥n", $$);
        return 0;

}
E:E' +' E {$$=$1+$3;}

|E' -' E {$$=$1-$3;}

|E' *' E {$$=$1*$3;}

|E' /' E {$$=$1/$3;}

|E'%' E {$$=$1・$3;}

|' (' E' )'  {$$=$2;}

| NUMBER {$$=$1;}

;

%%
void main()
{
printf("¥nEnter Any Arithmetic Expression:¥n");
yyparse();
```

```
}
```

```
void yyerror()
{
  printf("¥nInvalid¥n");
}
```

Output



Result

The program is executed and the output is obtained and verified.